COMPUTER VISION

Analisis Mendalam Topik Lightweight YOLO26n-Depth



Nama Dosen:

Prof. Dr. Eng. Fitri Utaminingrum, S.T., M.T.

Matakuliah & Kelas

Computer Vision - Kelas C

FAKULTAS ILMU KOMPUTER

PROGRAM STUDI TEKNIK KOMPUTER

UNIVERSITAS BRAWIJAYA

TAHUN 2026

KELOMPOK 1.1
Lightweight YOLO26-Depth

NAMA: Lutfi Maulana (NIM. 245150301111021)
TUGAS: Pengkajian arsitektur Dual-Head YOLO26n (Backbone, Neck, Detection & Depth Head), penentuan ruang lingkup dan batasan sistem, serta penyusunan alur runtime aplikasi desktop (PyTorch/Visual Studio)

NAMA : Muhammad Dzakwan Ikram (NIM. 245150301111019)
TUGAS: Penyusunan landasan teori object detection & depth estimation, penulisan konsep Multi-Task Learning dan ROI Mapping, serta analisis metadata dataset (SUN RGB-D dan COCO Mix).

1.1. YOLO

YOLO (You Only Look Once) adalah algoritma kecerdasan buatan yang dirancang khusus untuk melakukan *object detection* secara otomatis pada gambar maupun video. Berbeda dengan klasifikasi biasa yang hanya menebak isi gambar, YOLO bisa mengenali jenis objek sekaligus menandai lokasinya menggunakan *bounding box* (Redmon dkk., 2016)..

Prinsip kerjanya sesuai dengan namanya, yaitu memproses seluruh piksel gambar secara bersamaan hanya dalam satu kali penelusuran. Dengan cara ini YOLO jadi sangat cepat dan mampu memahami konteks visual secara utuh tanpa perlu memotong gambar menjadi bagian-bagian kecil terlebih dahulu.

1.2. YOLO26 dan Penerapan pada Proyek

Pada proyek ini, kami menggunakan algoritma YOLO26 versi Nano (yolo26n) karena ukurannya yang ringan dan prosesnya cepat. Secara bawaan, YOLO26 sebenarnya hanya bertugas mendeteksi jenis dan lokasi objek. Agar sistem ini juga bisa mengukur jarak, kami memakai pendekatan Multi-Task Learning dengan menggabungkan detektor yolo26n dan modul pengukur kedalaman (depth head) di dalam satu file model (multitask_best.pt). Dengan cara ini, komputer hanya perlu membaca gambar satu kali untuk menjalankan dua tugas sekaligus: mendeteksi objek dan membuat peta kedalaman (depth map).

Untuk menentukan jarak objek ke kamera, sistem menggunakan teknik Region of Interest (ROI) Mapping. Caranya sederhana: kotak deteksi (bounding box) hasil YOLO akan dicocokkan lokasinya dengan area yang sama di depth map. Nilai kedalaman di area tersebut kemudian diambil dan diubah menjadi angka jarak nyata dalam satuan meter.

Dalam pengerjaannya, proyek ini memiliki beberapa batasan sistem:

1. **Objek Dibatasi 4 Kelas**: Sistem difokuskan untuk mendeteksi orang, botol, laptop, dan HP. Keempat objek ini

dipilih karena merupakan benda yang paling sering berada di depan kamera pada area dalam ruangan.

2. Output Aplikasi: Hasil akhir yang ditampilkan di layar berupa *bounding box*, label nama objek, tampilan visual *depth map*, serta nilai jarak objek dalam satuan meter.
3. Lingkungan Pengujian: Aplikasi dieksekusi dan diuji murni menggunakan laptop pada lingkungan Visual Studio berbasis PyTorch.



Sumber: https://developers.google.com/ar/develop/java/depth/quickstart?hl=id

Gambar 1.1 Ilustrasi umum RGB dan monocular depth pada perangkat mobile; bukan tangkapan layar atau hasil pengujian aplikasi proyek.

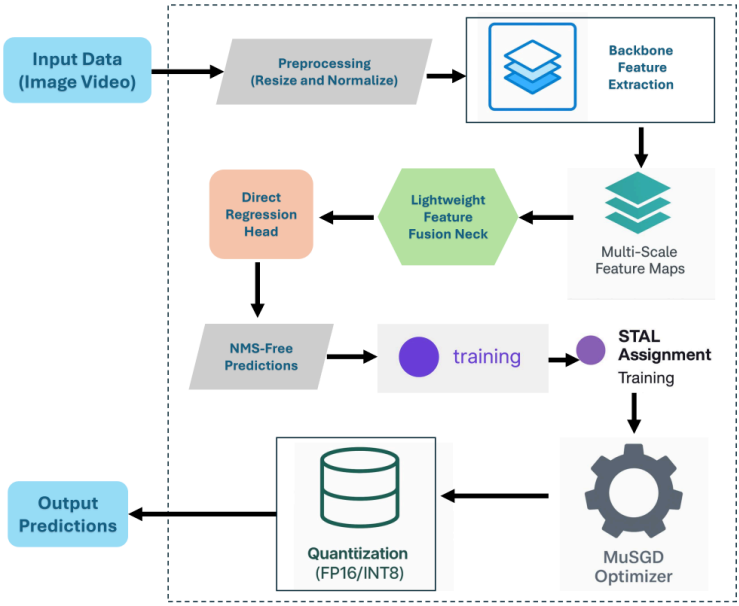
1.2.1 ARSITEKTUR YOLO V26

Secara sederhana, arsitektur YOLO26n terdiri dari tiga bagian utama:

- **Backbone**: Bertugas membaca dan memahami isi gambar, mulai dari garis, tekstur, hingga bentuk dasar objek.

- **Neck**: Menggabungkan informasi dari berbagai ukuran gambar agar objek berukuran kecil maupun besar dapat terdeteksi dengan baik.
- **Detection Head**: Mengolah hasil dari *Neck* untuk menebak nama objek sekaligus menentukan posisi kotaknya (*bounding box*) secara langsung.

Pada model *multitask* yang digunakan, hasil olahan dari bagian *Neck* dikirim ke dua saluran sekaligus: ke **Detection Head** untuk mengenali objek, dan secara bersamaan ke **Depth Head** untuk membuat peta kedalaman (*depth map*).



Sumber: https://learnopencv.com/yolov26-real-time-deployment/

Gambar 1.2 Referensi arsitektur YOLO26; bukan diagram lengkap depth head multitask runtime proyek.

1.3 DATASET

Pelatihan deteksi dan depth memerlukan citra serta label yang sesuai untuk tiap tugas. Dataset yang benar-benar digunakan harus mengikuti konfigurasi dan catatan run checkpoint, bukan daftar dataset contoh pada tinjauan pustaka.

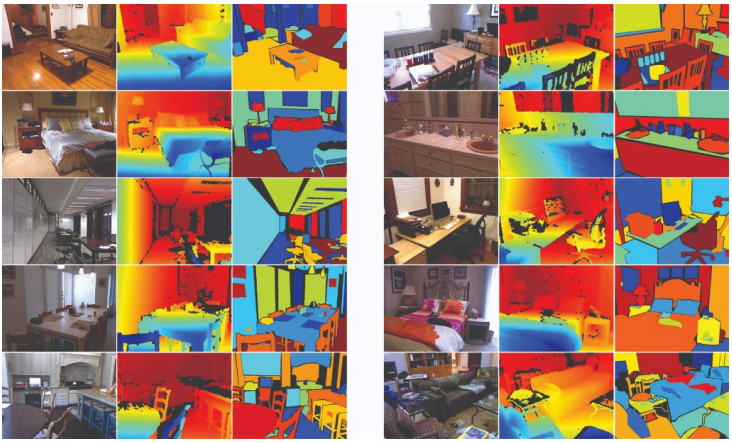
Metadata checkpoint multitask_best.pt mencatat LABEL_MODE=SUNRGBD, COCO_MIX=0.3, empat kelas person/laptop/cell phone/bottle, N_VAL=500, dan N_TEST=500. Metadata yang tersedia tidak cukup untuk mengklaim bahwa NYU Depth V2 atau KITTI dipakai pada run ini.

- SUN RGB-D: mode label depth yang dicatat pada konfigurasi checkpoint eksperimen ini. Detail subset dan pembagian berkas mengikuti konfigurasi/log run.

- COCO mix: konfigurasi run mencatat COCO_MIX=0.3 untuk campuran data deteksi. Ini bukan bukti bahwa COCO digunakan sebagai dataset depth atau detector runtime menggunakan kelas COCO lengkap.

- MS COCO (Common Objects in Context): dataset deteksi objek; konfigurasi run mencatat COCO_MIX=0.3. Bobot detector hasil akhir disesuaikan dengan empat kelas pada checkpoint.

Checkpoint berisi empat kelas target dan depth metric hasil pelatihan multitask. Klaim generalisasi ke semua lingkungan atau deployment smartphone memerlukan evaluasi tambahan.



Sumber: https://cs.nyu.edu/~fergus/datasets/nyu_depth_v2.html

Gambar 1.4 Contoh dari NYU Depth V2 sebagai ilustrasi dataset depth umum; bukan dataset atau hasil inferensi yang diklaim untuk checkpoint ini.

KELOMPOK 1.1
Lightweight YOLO26-Depth

NAMA: Lutfi Maulana (NIM. 245150301111021)
TUGAS: Merancang ringkasan alur kerja pipeline baseline serta mengintegrasikan kode pemanggilan model PyTorch dan arsitektur YOLO26nDepth.

NAMA : Muhammad Dzakwan Ikram (NIM. 245150301111019)
TUGAS: Mengimplementasikan kode penanganan stream kamera dan visualisasi serta menyusun pembedahan detail logika dan fungsi program.

2.1 Deskripsi Alur Pipeline Baseline

Aplikasi kamera ini berjalan menggunakan satu berkas model multitask PyTorch (.pt) untuk mendeteksi objek sekaligus membuat peta kedalaman (depth map) dari webcam secara langsung. Seluruh proses inferensi dieksekusi murni pada lingkungan PyTorch.

Alur Kerja Program:

1. Ambil Gambar: Mengambil frame video dari webcam.
2. Olah Gambar: Mengatur ukuran gambar (letterbox) dan menyelaraskan nilai pikselnya.
3. Proses Model: Memproses deteksi objek dan peta kedalaman secara bersamaan di PyTorch.
4. Kembalikan Skala: Menyesuaikan kembali posisi kotak deteksi dan peta kedalaman ke ukuran layar asli.
5. Penyempurnaan: Menghaluskan tampilan kedalaman antar-frame dan menerapkan rumus kalibrasi.
6. Tampilkan Hasil: Menampilkan kotak deteksi, label objek, jarak dalam meter, dan visualisasi warna kedalaman pada layar.

2.2 Kode Program Baseline

Potongan kode berikut merupakan ringkasan fungsi utama dari aplikasi kamera real-time yang dijalankan melalui Visual Studio. Implementasi lengkapnya terbagi di dalam berkas multitask_pt.py dan main.py.

```
import cv2
# Memuat modul dari multitask_pt.py dan utils.py
from multitask_pt import load_multitask_torch_model
from utils import DepthEMASmoother, calibrate_depth_map,
get_depth_in_bbox, normalize_depth_for_display

# 1. Inisialisasi Model Multitask (.pt)
# Memuat checkpoint model PyTorch secara strict (Backbone +
Detection Head + Depth Head)
model = load_multitask_torch_model("multitask_best.pt")
```

```
smoother = DepthEMASmoother(alpha=0.2)

# --- Loop Utama Kamera (main.py) ---
while True:
   ret, frame = cap.read()
   if not ret:
      break

   # 2. Inferensi Multitask (Eksekusi di multitask_pt.py)
   # Menghasilkan bounding box [x1, y1, x2, y2, conf, cls] dan
depth map (meter)
   detections, depth_m = model.predict(frame)

   # 3. Smoothing & Kalibrasi Depth Map
   # Meredam fluktuasi antar-frame dan menerapkan
skala/offset empiris
   depth_smoothed = smoother.update(depth_m)
   depth_calibrated = calibrate_depth_map(depth_smoothed,
depth_scale=0.9935897436, depth_offset_m=-1.0)

   # 4. Pengukuran Jarak Objek (ROI Mapping)
   for det in detections:
      x1, y1, x2, y2, conf, cls_id = det
      # Mengambil median depth pada patch 15x15 piksel di
pusat bounding box
      distance_m = get_depth_in_bbox(
         depth_m=depth_calibrated,
         bbox=(x1, y1, x2, y2),
         patch_size=15
      )

   # 5. Konversi Visualisasi Heatmap Kedalaman (Khusus
Tampilan UI)
   depth_color = cv2.applyColorMap(
      normalize_depth_for_display(depth_calibrated),
      cv2.COLORMAP_INFERNO
   )

   # Tampilkan hasil pada layar (main.py)
   cv2.imshow("Multi-Task YOLO26n & Depth Estimation",
frame)
```

```
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
```

Sumber: Penulis

2.3 Penjelasan Logika dan Fungsi Program

Bagian ini menjelaskan fungsi teknis dari setiap komponen utama dalam skrip multitask_pt.py dan main.py dengan kalimat yang lebih sederhana:

- **Pengolahan Awal (Preprocessing)**: Gambar dari kamera disesuaikan ukurannya (*letterbox* 480 × 640), diubah ke format warna RGB, dan diselaraskan nilai pikselnya agar siap dibaca oleh model.
- **Proses Model (Inferensi Multitask)**: Model menjalankan dua tugas sekaligus dalam satu waktu: deteksi objek YOLO26n (menghasilkan lokasi kotak dan nama objek) serta *Depth Head* (menghasilkan peta kedalaman dalam satuan meter).
- **Penyesuaian Skala (Postprocessing)**: Posisi kotak deteksi dan peta kedalaman dikembalikan ukurannya agar sesuai dengan tampilan asli layar kamera.
- **Penghalusan dan Kalibrasi**: Angka kedalaman dihaluskan antar-*frame* agar tidak goyang, lalu disesuaikan dengan rumus kalibrasi empiris agar pembacaan jaraknya lebih stabil.
- **Pengukuran Jarak Objek (ROI Mapping)**: Sistem mengambil nilai tengah (*median*) kedalaman pada area kotak 15 × 15 piksel di pusat objek untuk menentukan jarak fisik nyata dalam satuan meter.
- **Visualisasi Warna (Heatmap)**: Peta kedalaman diubah menjadi warna gradien *INFERNO*. Warna ini **hanya untuk tampilan visual di layar (UI)** agar nyaman dilihat dan tidak dipakai dalam perhitungan angka jarak.
- **Tampilan dan Kontrol**: Layar dapat menampilkan gambar secara berdampingan atau ditumpuk (*overlay*). Tombol **s** mengganti mode tampilan, **d** mengaktifkan *overlay*, **q** menghentikan program, dan klik *mouse* digunakan untuk membaca jarak di titik tertentu.
- **Pemuatan Model (Checkpoint)**: Fungsi load_multitask_torch_model() memastikan berkas bobot

multitask_best.pt lengkap dan sesuai sebelum program kamera dijalankan.

KELOMPOK 1.1
Lightweight YOLO26-Depth

NAMA: Lutfi Maulana (NIM. 245150301111021)
TUGAS: Menyusun ringkasan konfigurasi spesifikasi model baseline YOLO26n (compound scaling, backbone, neck, head) serta menganalisis diagram alur arsitektur dan titik modifikasinya (Sub-bab 3.1).

NAMA : Muhammad Dzakwan Ikram (NIM. 245150301111019)
TUGAS: Memverifikasi kesesuaian implementasi kode program PyTorch dengan rancangan arsitektur, mulai dari blok konvolusi dasar hingga integrasi modul DepthHead dan fungsi forward (Sub-bab 3.2).

3.1 Spesifikasi Arsitektur Baseline (YOLO26n Multitask)

Model yang dibangun berbasis arsitektur YOLO26n Multitask, yang mengombinasikan shared encoder (Backbone dan Neck) dengan dua head terpisah (Dual-Head), yaitu Detection Head dan Depth Head. Tabel 3.1 merangkum konfigurasi spesifikasi arsitektur baseline acuan beserta modul penyesuaian multitask dengan empat kelas target yang diimplementasikan pada sistem runtime.

Tabel 3.1 Ringkasan Konfigurasi Arsitektur Model Multitask YOLO26n

Komponen	Layer / Sumber	Modul Utama	Output / Fungsi
Compound Scaling	Varian YOLO26n	depth = 0.50, width = 0.25, max channels = 1024	Varian model nano yang efisien (260 layer, ~2.57M parameter, 6.1 GFLOPs) untuk eksekusi real-time pada perangkat desktop.
Backbone	Layer 0–10	Conv, C3k2, SPPF, C2PSA	Ekstraksi fitur hierarkis dari skala P1/2 hingga P5/32
Fitur P3	Layer 3–4 (digunakan kembali di Neck)	Conv dan C3k2	Menyediakan fitur spasial detail untuk deteksi objek berukuran kecil (small objects)
Fitur P4	Layer 5–6 (digunakan kembali	Conv dan C3k2	Menyediakan fitur tingkat menengah

	di Neck)		untuk objek berukuran sedang (medium objects)
Fitur P5	Layer 7–10	Conv, C3k2, SPPF, C2PSA	Menyediakan fitur semantik tingkat tinggi untuk objek berukuran besar (large objects)
Neck	Layer 11–22	Upsample, Concat, Conv, C3k2	Menggabungkan (aggregate) fitur multi-skala dari alur backbone untuk diteruskan ke Dual-Head.
Detection Head	Layer 23 (dari Layer 16, 19, 22)	Detect	Memprediksi bounding box dan label klasifikasi 4 kelas target (person, bottle, laptop, cell phone) pada skala P3/8, P4/16, dan P5/32.
Depth Head	Percabangan dari Neck (P3, P4, P5)	Conv, Bilinear Interpolation, Feature Fusion	Mengolah fitur multi-skala menjadi depth map satu kanal (satuan meter) untuk estimasi jarak objek.

Sumber: Ardhi et al. (2026) dan Pengembangan Penulis

3.1.1 Skala Komputasi (Compound Scaling)

Arsitektur YOLO26n dikonfigurasi menggunakan depth factor sebesar 0.50 dan width factor sebesar 0.25 dengan batas saluran maksimum 1024 channel. Konfigurasi ini menghasilkan model yang efisien dan ringan dengan total 260 layer, 2.572.280 parameter, serta beban komputasi sebesar 6.1 GFLOPS, sehingga ideal untuk dijalankan secara real-time pada lingkungan desktop.

3.1.2 Ekstraksi Fitur (Backbone):

Bagian backbone beroperasi pada Layer 0 hingga 10 menggunakan kombinasi konvolusi standar (Conv), blok C3k2, SPPF (Spatial Pyramid Pooling - Fast), dan C2PSA (Cross Stage Partial with Attention). Backbone mengekstrak representasi citra secara hierarkis yang terbagi menjadi tiga tingkatan resolusi:

- Fitur P3 (Layer 3–4): Mempertahankan informasi spasial detail yang berguna untuk mendeteksi objek berukuran kecil.
- Fitur P4 (Layer 5–6): Mengambil fitur tekstur tingkat menengah untuk objek berukuran sedang.
- Fitur P5 (Layer 7–10): Menangkap informasi semantik tingkat tinggi untuk membedakan objek berukuran besar.

Penjelasan fungsi modul penyusun *backbone*:

- **Conv (Convolution)**: Melakukan *downsampling* (memperkecil resolusi spasial) sekaligus mengekstrak fitur visual dasar.
- **C3k2**: Blok pemroses utama berbasis *Cross Stage Partial* (CSP) yang efisien mengekstrak fitur tanpa membengkakkan beban memori.
- **SPPF**: Menggabungkan peta fitur dari berbagai skala *pooling* secara cepat untuk memperluas area pandang (*receptive field*) model.

- **C2PSA**: Blok *attention* bawaan YOLO26n yang berfungsi menangkap hubungan spasial antarfitur sebelum diteruskan ke bagian Neck.

3.1.3 Penggabungan Fitur Multi-Skala (Neck)

Terletak pada Layer 11 hingga 22, komponen Neck memanfaatkan fungsi Upsample, Concat, Conv, dan C3k2 melalui arsitektur FPN (Feature Pyramid Network) + PAN (Path Aggregation Network). Neck menggabungkan peta fitur dari berbagai resolusi yang dihasilkan oleh backbone, memastikan bahwa fitur spasial detail dan fitur semantik kaya dapat saling melengkapi sebelum disalurkan ke Dual-Head.

3.1.4 Prediksi Akhir (Detection Head & Task Setting):

Sistem menggunakan pendekatan *Dual-Head* setelah bagian Neck:

1. **Detection Head**: Menerima fitur dari layer 16, 19, dan 22 pada skala P3/8, P4/16, dan P5/32 untuk menghasilkan prediksi *bounding box* dan skor klasifikasi secara *end-to-end* untuk 4 kelas target (*person*, *bottle*, *laptop*, *cell phone*).
2. **Depth Head**: Menerima peta fitur dari Neck untuk diproyeksikan dan digabungkan (*fusion*) menjadi peta kedalaman (*depth map*) satu kanal beresolusi penuh dalam satuan meter aktual.

3.1.5 Alur Arsitektur dan Modifikasi Model

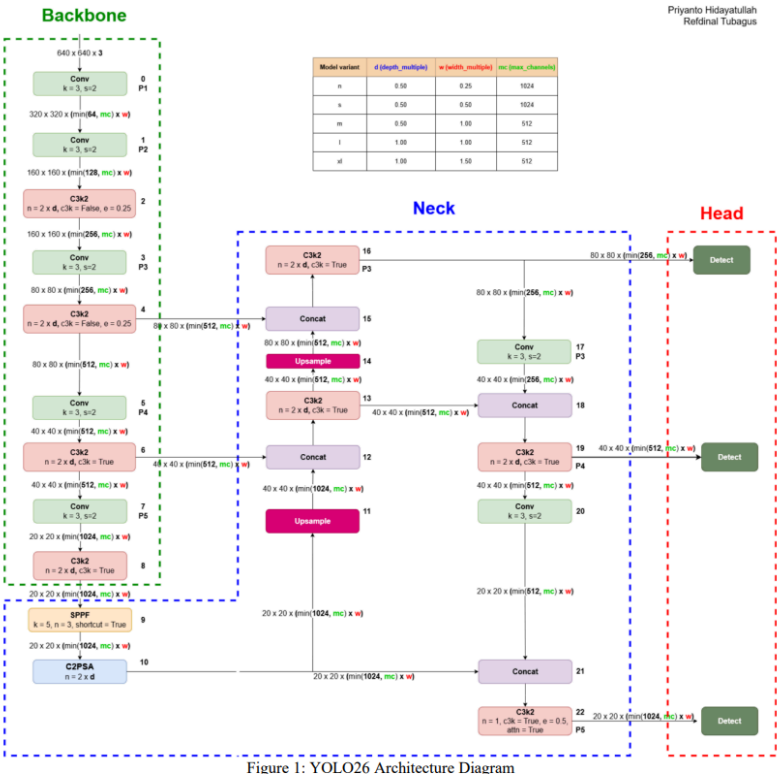
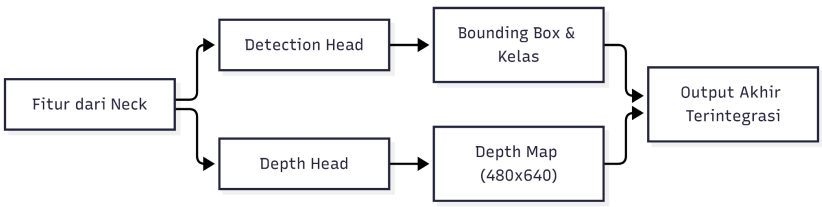


Figure 1: YOLO26 Architecture Diagram

Sumber: Hidayatullah dan Tubagus (2026)

Gambar 3.1 Arsitektur Utama YOLO26 (Backbone & Neck)



Sumber: Penulis

Gambar 3.2 Diagram Percabangan Dual-Head Model Multitask

Diagram arsitektur pada Gambar 3.1 menggambarkan alur perjalanan data dari citra masukan hingga menghasilkan dua jenis output secara serentak (multitask):

- Backbone (Layer 0–10):

    - Frame masukan *webcam* beresolusi asli diolah (*letterbox*) menjadi resolusi standar 480 × 640 piksel.
    - **Stem Layer (Layer 0–1)**: Menggunakan Konvolusi 3 × 3 (Conv) dengan *stride* 2 untuk mereduksi resolusi spasial awal sekaligus mengekstrak fitur visual dasar.
    - **Ekstraksi Hierarkis (Layer 2, 4, 6, 8)**: Memanfaatkan modul C3k2 untuk mengekstrak fitur visual abstrak secara efisien dari tingkatan skala $P2/4$ hingga $P5/32$.
    - **SPPF & C2PSA (Layer 9–10)**: Modul SPPF memperluas area pandang (*receptive field*) fitur, diikuti oleh modul *self-attention* bawaan C2PSA untuk memperkuat hubungan spasial antarfitur semantik sebelum disalurkan ke Neck.
- Neck / FPN + PAN (Layer 11–22) :
    - Komponen Neck menggabungkan peta fitur dari *backbone* melalui dua alur utama: *Feature Pyramid Network* (FPN) secara *top-down* dan *Path Aggregation Network* (PAN) secara *bottom-up*.
    - Proses penggabungan (*concat*) dan penyesuaian ukuran (*upsample*/*downsample*) menghasilkan tiga tingkatan peta fitur kaya informasi spasial dan semantik, yaitu pada skala $P3/8$ (deteksi kecil), $P4/16$ (deteksi sedang), dan $P5/32$ (deteksi besar).

Gambar 3.2 menjelaskan bagaimana tiga tingkatan peta fitur ($P3, P4, P5$) keluaran dari *Neck* dialirkan secara bersamaan (*simultaneous forward pass*) ke dua modul *head* independen:

- Detection Head:
    - Menerima peta fitur multi-skala dari Neck untuk memprediksi lokasi *bounding box* (x_1, y_1, x_2, y_2), skor *confidence*, dan probabilitas klasifikasi untuk 4 kelas target (*person*, *bottle*, *laptop*, *cell phone*).

    - Eksekusi dilakukan secara *end-to-end* (bebas operasi NMS) untuk memastikan latensi inferensi tetap rendah.
- Depth Head:
    - Menerima peta fitur dari Neck untuk diproyeksikan (256 kanal), digabungkan (*feature fusion*), dan disesuaikan ukurannya melalui interpolasi *bilinear*.
    - Menghasilkan *depth map* satu kanal beresolusi penuh (480×640) dengan nilai kedalaman fisik langsung dalam satuan meter metrik.
- Output Akhir Terintegrasi:
    - Hasil olahan *bounding box* dan *depth map* digabungkan di tingkat aplikasi *runtime*.
    - Sistem melakukan *ROI Mapping* (15×15 *patch* di pusat *bounding box*) untuk mengekstrak nilai jarak metrik objek, lalu menampilkan *heatmap* kedalaman beserta label deteksi pada antarmuka *webcam* secara *real-time*.

3.2 Penjelasan Kesesuaian Program dengan Arsitektur

Subbab ini menguraikan struktur modul PyTorch dan blok-blok logika yang membentuk arsitektur jaringan YOLO26n Multitask. Seluruh modul dirancang secara teratur untuk mendukung eksekusi Shared Encoder (Backbone dan Neck) hingga percabangan keluaran Dual-Head (Detection Head dan Depth Head).

1. ConvBlock

ConvBlock merupakan blok pembangun dasar yang digunakan di seluruh bagian arsitektur. Blok ini terdiri atas konvolusi 2D, Batch Normalization, dan fungsi aktivasi SiLU (Sigmoid Linear Unit). Konvolusi mengekstrak karakteristik visual dasar, Batch Normalization menjaga kestabilan distribusi nilai fitur, sedangkan aktivasi SiLU memberikan kemampuan non-linearitas agar model dapat mempelajari representasi fitur yang kompleks.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def autopad(k, p=None, d=1):
  if d > 1:
    k = d * (k - 1) + 1 if isinstance(k, int) else [d * (x - 1) + 1 for x
in k]
  if p is None:
    p = k // 2 if isinstance(k, int) else [x // 2 for x in k]
  return p

class ConvBlock(nn.Module):
  def __init__(self, c1, c2, k=1, s=1, p=None, g=1, d=1, act=True):
    super().__init__()
    self.conv = nn.Conv2d(c1, c2, k, s, autopad(k, p, d),
groups=g, dilation=d, bias=False)
    self.bn = nn.BatchNorm2d(c2)
    self.act = nn.SiLU() if act is True else (act if isinstance(act,
nn.Module) else nn.Identity())

  def forward(self, x: torch.Tensor) -> torch.Tensor:
    return self.act(self.bn(self.conv(x)))
```

2. Bottleneck

Modul Bottleneck terdiri atas dua lapisan ConvBlock berurutan dengan koneksi residual (shortcut). Koneksi residual memungkinkan informasi fitur awal diteruskan secara langsung ke output, sehingga mempermudah aliran gradien saat pelatihan dan mencegah masalah vanishing gradient. Blok ini menjadi komponen utama penyusun modul C3k2.

```
class Bottleneck(nn.Module):
  def __init__(self, c1, c2, shortcut=True, g=1, k=(3, 3), e=0.5):
    super().__init__()
    c_ = int(c2 * e)
    self.cv1 = ConvBlock(c1, c_, k[0], 1)
    self.cv2 = ConvBlock(c_, c2, k[1], 1, g=g)
    self.add = shortcut and c1 == c2
```

```
  def forward(self, x: torch.Tensor) -> torch.Tensor:
    return x + self.cv2(self.cv1(x)) if self.add else
self.cv2(self.cv1(x))
```

3. C3k2

Modul C3k2 merupakan blok ekstraksi fitur berbasis Cross Stage Partial (CSP). Fitur masukan dibagi menjadi dua jalur: satu jalur diproses langsung melalui konvolusi, sedangkan jalur lainnya diproses melalui deretan blok Bottleneck. Kedua jalur kemudian digabungkan (concatenation) pada dimensi channel (dim=1) dan diproyeksikan kembali, sehingga menghasilkan representasi fitur yang kaya tanpa membengkakkan beban komputasi.

```
class C3k2(nn.Module):
  def __init__(self, c1, c2, n=1, c3k=False, e=0.5, g=1,
shortcut=True):
    super().__init__()
    c_ = int(c2 * e)
    self.cv1 = ConvBlock(c1, c_, 1, 1)
    self.cv2 = ConvBlock(c1, c_, 1, 1)
    self.m = nn.ModuleList([Bottleneck(c_, c_, shortcut, g, k=(3,
3), e=1.0) for _ in range(n)])
    self.cv3 = ConvBlock(2 * c_, c2, 1, 1)

  def forward(self, x: torch.Tensor) -> torch.Tensor:
    y1 = self.cv1(x)
    y2 = self.cv2(x)
    for m in self.m:
      y2 = m(y2)
    return self.cv3(torch.cat((y1, y2), dim=1))
```

4. C2PSA

C2PSA (Cross Stage Partial with Attention) merupakan modul attention bawaan (stock) pada arsitektur YOLO26n. Modul ini menerapkan Adaptive Average Pooling dan konvolusi berbobot sigmoid untuk memberikan pembobotan antar channel (mekanisme spatial & channel attention), sehingga model fokus

pada area fitur yang paling relevan sebelum disalurkan ke bagian Neck.

```
class C2PSA(nn.Module):
    def __init__(self, c1, c2, n=1, e=0.5):
        super().__init__()
        c_ = int(c2 * e)
        self.cv1 = ConvBlock(c1, 2 * c_, 1, 1)
        self.cv2 = ConvBlock(2 * c_, c2, 1, 1)
        self.attn = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Conv2d(c2, c2 // 4, 1),
            nn.SiLU(),
            nn.Conv2d(c2 // 4, c2, 1),
            nn.Sigmoid()
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.cv1(x)
        return self.cv2(x * self.attn(x))
```

5. SPPF

Modul SPPF berfungsi untuk memperluas area pandang (receptive field) model secara efisien. Dengan melakukan operasi max pooling bertahap menggunakan ukuran kernel konstan (5 x 5), SPPF mampu menangkap informasi konteks multi-skala dan menggabungkannya dengan fitur awal tanpa menurunkan resolusi spasial.

```
class SPPF(nn.Module):
    def __init__(self, c1, c2, k=5):
        super().__init__()
        c_ = c1 // 2
        self.cv1 = ConvBlock(c1, c_, 1, 1)
        self.cv2 = ConvBlock(c_ * 4, c2, 1, 1)
        self.m = nn.MaxPool2d(kernel_size=k, stride=1, padding=k
// 2)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.cv1(x)
```

```
        y1 = self.m(x)
        y2 = self.m(y1)
        y3 = self.m(y2)
        return self.cv2(torch.cat((x, y1, y2, y3), dim=1))
```

6. Backbone

Backbone bertanggung jawab melakukan ekstraksi fitur hierarkis dari gambar masukan. Proses dimulai dari konvolusi awal (stem) dengan stride 2, diikuti oleh kombinasi modul ConvBlock, C3k2, SPPF, dan C2PSA. Backbone menghasilkan tiga tingkatan peta fitur utama:

- P3: Resolusi spasial tinggi untuk deteksi objek berukuran kecil.
- P4: Resolusi menengah untuk deteksi objek berukuran sedang.
- P5: Resolusi spasial rendah dengan pemahaman semantik tingkat tinggi untuk objek besar.

```
class Backbone(nn.Module):
    def __init__(self, c_stem=16, c_p2=32, c_p3=64, c_p4=128,
c_p5=256, n_repeat=1):
        super().__init__()
        self.stem = ConvBlock(3, c_stem, k=3, s=2)
        self.l2 = C3k2(c_stem, c_p2, n=n_repeat)
        self.l3 = ConvBlock(c_p2, c_p3, k=3, s=2)
        self.l4 = C3k2(c_p3, c_p3, n=n_repeat)
        self.l5 = ConvBlock(c_p3, c_p4, k=3, s=2)
        self.l6 = C3k2(c_p4, c_p4, n=n_repeat)
        self.l7 = ConvBlock(c_p4, c_p5, k=3, s=2)
        self.l8 = C3k2(c_p5, c_p5, n=n_repeat)
        self.l9 = SPPF(c_p5, c_p5, k=5)
        self.l10 = C2PSA(c_p5, c_p5)

    def forward(self, x: torch.Tensor):
        x = self.stem(x)
        x = self.l2(x)
        x = self.l3(x)
```

```
        p3 = self.l4(x)
        x = self.l5(p3)
        p4 = self.l6(x)
        x = self.l7(p4)
        x = self.l8(x)
        x = self.l9(x)
        p5 = self.l10(x)
        return p3, p4, p5
```

7. Neck FPN dan PAN

Komponen Neck menggabungkan fitur multi-skala dari backbone menggunakan gabungan arsitektur Feature Pyramid Network (FPN) dan Path Aggregation Network (PAN). Alur FPN menyebarkan informasi semantik secara top-down (P5 ke P3), sedangkan PAN menyalurkan kembali informasi posisi spasial presisi secara bottom-up (P3 ke P5). Hasil akhirnya berupa tiga peta fitur kaya konteks (p3_out, p4_out, p5_out).

```
class Neck(nn.Module):
    def __init__(self, c_p3=64, c_p4=128, c_p5=256):
        super().__init__()
        self.up_p5_to_p4 = nn.Upsample(scale_factor=2,
mode='nearest')
        self.cv_p4 = C3k2(c_p5 + c_p4, c_p4)
        self.up_p4_to_p3 = nn.Upsample(scale_factor=2,
mode='nearest')
        self.cv_p3 = C3k2(c_p4 + c_p3, c_p3)

        self.down_p3_to_p4 = ConvBlock(c_p3, c_p3, k=3, s=2)
        self.pan_p4 = C3k2(c_p3 + c_p4, c_p4)
        self.down_p4_to_p5 = ConvBlock(c_p4, c_p4, k=3, s=2)
        self.pan_p5 = C3k2(c_p4 + c_p5, c_p5)

    def forward(self, p3, p4, p5):
        f_p5_up = self.up_p5_to_p4(p5)
        p4_fused = self.cv_p4(torch.cat([f_p5_up, p4], dim=1))
        f_p4_up = self.up_p4_to_p3(p4_fused)
        p3_out = self.cv_p3(torch.cat([f_p4_up, p3], dim=1))
```

```
        p3_down = self.down_p3_to_p4(p3_out)
        p4_out = self.pan_p4(torch.cat([p3_down, p4_fused],
dim=1))
        p4_down = self.down_p4_to_p5(p4_out)
        p5_out = self.pan_p5(torch.cat([p4_down, p5], dim=1))

        return p3_out, p4_out, p5_out
```

8. Detection Head

DetectionHead bertugas memprediksi *bounding box* dan label kelas objek pada tiga skala berbeda ($P3/8$, $P4/16$, $P5/32$). Setiap *head* memiliki dua cabang konvolusi terpisah: cabang regresi (reg) untuk memprediksi koordinat lokasi (x_1, y_1, x_2, y_2) dan cabang klasifikasi (cls) untuk menentukan probabilitas 4 kelas target (*person*, *bottle*, *laptop*, *cell phone*).

```
class DetectionHead(nn.Module):
    def __init__(self, in_ch: int, num_classes: int = 4):
        super().__init__()
        self.reg = nn.Sequential(
            ConvBlock(in_ch, in_ch, k=3, s=1),
            nn.Conv2d(in_ch, 4, kernel_size=1)
        )
        self.cls = nn.Sequential(
            ConvBlock(in_ch, in_ch, k=3, s=1),
            nn.Conv2d(in_ch, num_classes, kernel_size=1)
        )

    def forward(self, x: torch.Tensor):
        return self.reg(x), self.cls(x)
```

9. DepthHead

DepthHead memproyeksikan peta fitur multi-skala dari Neck ($P3, P4, P5$) ke jumlah *channel* yang setara, lalu menyesuaikan ukurannya ke dimensi target menggunakan interpolasi *bilinear*. Ketiga peta fitur digabungkan (*fusion*) dan

diproses oleh blok regresi untuk menghasilkan peta kedalaman (*depth map*) satu kanal dengan nilai meter metrik aktual.

```
class DepthHead(nn.Module):
    def __init__(self, channels=(64, 128, 256), out_ch=32):
        super().__init__()
        self.conv_p3 = ConvBlock(channels[0], out_ch, k=1, s=1)
        self.conv_p4 = ConvBlock(channels[1], out_ch, k=1, s=1)
        self.conv_p5 = ConvBlock(channels[2], out_ch, k=1, s=1)
        self.refine = nn.Sequential(
            ConvBlock(out_ch * 3, 64, k=3, s=1),
            nn.Conv2d(64, 1, kernel_size=1)
        )

    def forward(self, p3, p4, p5, target_size):
        up_p3 = F.interpolate(self.conv_p3(p3), size=target_size,
mode='bilinear', align_corners=False)
        up_p4 = F.interpolate(self.conv_p4(p4), size=target_size,
mode='bilinear', align_corners=False)
        up_p5 = F.interpolate(self.conv_p5(p5), size=target_size,
mode='bilinear', align_corners=False)
        fused = torch.cat([up_p3, up_p4, up_p5], dim=1)
        return self.refine(fused)
```

10. Forward Model

Fungsi forward() mengatur alur pemrosesan terintegrasi dari masukan citra hingga keluaran Dual-Head. Gambar masukan pertama kali diekstraksi oleh Backbone, digabungkan oleh Neck, lalu disalurkan secara paralel ke tiga modul DetectionHead dan satu modul DepthHead.

```
class YOLO26Multitask(nn.Module):
    def __init__(self, num_classes=4):
        super().__init__()
        self.backbone = Backbone()
        self.neck = Neck()
        self.det_p3 = DetectionHead(64, num_classes)
        self.det_p4 = DetectionHead(128, num_classes)
        self.det_p5 = DetectionHead(256, num_classes)
        self.depth_head = DepthHead(channels=(64, 128, 256))
```

```
  def forward(self, x: torch.Tensor):
    target_size = (x.shape[2], x.shape[3])

    p3, p4, p5 = self.backbone(x)
    p3_out, p4_out, p5_out = self.neck(p3, p4, p5)

    p3_reg, p3_cls = self.det_p3(p3_out)
    p4_reg, p4_cls = self.det_p4(p4_out)
    p5_reg, p5_cls = self.det_p5(p5_out)
    depth_map = self.depth_head(p3_out, p4_out, p5_out,
target_size)

    return {
      'p3_bbox': p3_reg, 'p3_cls': p3_cls,
      'p4_bbox': p4_reg, 'p4_cls': p4_cls,
      'p5_bbox': p5_reg, 'p5_cls': p5_cls,
      'depth': depth_map
    }
```

11. Output Model

Model mengembalikan luaran dalam bentuk kamus (*dictionary*) PyTorch Tensor yang berisi hasil prediksi lokasi objek, skor kelas, dan estimasi kedalaman secara terpisah:

- p3_bbox, p4_bbox, p5_bbox: Tensor regresi lokasi *bounding box* pada skala $P3$, $P4$, $P5$.
- p3_cls, p4_cls, p5_cls: Tensor skor klasifikasi untuk 4 kelas target (*person*, *bottle*, *laptop*, *cell phone*).
- depth: Tensor peta kedalaman satu kanal beresolusi 480×640 piksel dengan estimasi jarak metrik dalam satuan meter.

Struktur keluaran terpisah ini memungkinkan aplikasi *runtime* mengeksekusi post-processing deteksi dan *ROI Mapping* kedalaman secara efisien dalam satu kali *forward pass*.

3.3 Mekanisme Kerja Dual-Head (Detection & Depth Estimation)

Bagian ini menguraikan mekanika pemrosesan data secara rinci pada cabang Dual-Head, mencakup metode dekode koordinat bounding box tanpa NMS (anchor-free), kalkulasi konversi peta kedalaman ke satuan meter metrik, serta teknik ekstraksi jarak objek berbasis ROI Mapping.

3.3.1 Detection Head: Dekode Bounding Box dan Klasifikasi Anchor-Free

3.3.2 Depth Head: Estimasi Kedalaman Metrik dan ROI Mapping 15x15

KELOMPOK 1.1
Lightweight YOLO26-Depth

NAMA: Lutfi Maulana (NIM. 245150301111021)
TUGAS: Mengidentifikasi dan membedah kelemahan teknis model baseline serta merumuskan rekomendasi arah perbaikan (Sub-bab 4.2 dan 4.3).

NAMA : Muhammad Dzakwan Ikram (NIM. 245150301111019)
TUGAS: Menyusun evaluasi kinerja kuantitatif model, analisis visual peta kedalaman, dan tabel ringkasan metrik (Sub-bab 4.1).

4.1 Evaluasi Kinerja Kuantitatif dan Analisis Visualisasi Output



Sumber: Penulis; contoh peta depth bersifat kualitatif.

Gambar 4.1 Hasil Output Depth Map

Tabel 4.1 Metrik Validasi yang Tersimpan pada Checkpoint Multitask (Epoch 10)

Metrik Evaluasi	Nilai	Keterangan

	Validasi	
delta1	0,7803 (78,03%)	Proporsi piksel valid dengan rasio kedalaman < 1,25 pada validasi checkpoint.
Abs Rel	0,1586 (15,86%)	Rata-rata kesalahan relatif kedalaman; nilai lebih kecil lebih baik.
RMSE	0,5989 m	Root mean squared error depth pada validasi checkpoint.
mAP50-95	0,1299 (12,99%)	Metrik deteksi mean Average Precision pada rentang IoU 0,50-0,95.

Sumber: metadata checkpoint multitask_best.pt, epoch 10; run 20261013-163528-539afa.

4.2 Analisis Kelemahan dan Keterbatasan Model Baseline

4.3 Rekomendasi Solusi dan Arah Perbaikan

4.4 Analisis Komparatif dan Keterbatasan Arsitektur Kustom

Sumber: Penulis

Gambar 4.2 Perbandingan Visual Output Inferensi Real-Time Model Kustom (Atas) dan Model Bawaan (Bawah)

DAFTAR PUSTAKA

Ardhi, O.D.W., Hartono, R. dan Yoeseph, N.M. (2026) 'Plant Disease Object Detection on PlantDoc Using YOLO26n', *SISFO Jurnal Ilmiah*, 10(1), hlm. 187–197. doi: 10.29103/sisfo.v10i1.27063.

Burukanli, M., Cibuk, M. dan Ari, D. (2026) 'Attention-Enhanced YOLOv26 with Tree-Structured Parzen Estimator Optimization for Robust Dental Surgical Tool Detection', *Applied Sciences*, 16(15), hlm. 7654. doi: 10.3390/app16157654.

Hidayatullah, P. dan Tubagus, R. (2026) "YOLO26: A Comprehensive Architecture Overview and Key Improvements", Preprint Paper, Politeknik Negeri Bandung & Stunning Vision AI.

Liu, S., Chen, R., Ye, M., Luo, J., Yang, D. dan Dai, M. 2024. EcoDetect-YOLO: A Lightweight, High-Generalization Methodology for Real-Time Detection of Domestic Waste Exposure in Intricate Environmental Landscapes. Sensors, 24(14), hal. 4666. https://doi.org/10.3390/s24144666

Ning, J., Liu, B., Zhou, B., Zhang, J., Ding, Y., Li, Y., Gao, Y. dan Ma, Y. 2026. An Attention-Guided Lightweight YOLO11n with Ghost Convolution for Pediatric Tooth Detection in Panoramic Radiographs. https://doi.org/10.18280/ts.430218

Redmon, J., Divvala, S., Girshick, R., dan Farhadi, A., 2016. You only look once: Unified, real-time object detection. Dalam Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788

Ultralytics. 2026. *YOLO26*. Ultralytics Documentations. Tersedia di: https://docs.ultralytics.com/models/yolo26

Yuan, Z., Tang, X., Ning, H. dan Yang, Z. 2024. LW-YOLO: Lightweight Deep Learning Model for Fast and Precise Defect Detection in Printed Circuit Boards. *Symmetry*, 16(4), hal. 418. https://doi.org/10.3390/sym16040418

Zare, O., Beigzadeh, M., Bzuayene, A.A., Arslan, E., Haghvirdizadeh, H., Rad, S.Y.B., Abbasi, A., Joloudari, J.H. dan Gaftandzhieva, S. 2026. LightAttn-YOLO-V8: an efficient colorectal polyp detection with

lightweight attention mechanism based on YOLO. Scientific Reports. https://doi.org/10.1038/s41598-026-35390-z

1.1. YOLO

YOLO (You Only Look Once) adalah algoritma kecerdasan buatan yang dirancang khusus untuk melakukan *object detection* secara otomatis pada gambar maupun video. Berbeda dengan klasifikasi biasa yang hanya menebak isi gambar, YOLO bisa mengenali jenis objek sekaligus menandai lokasinya menggunakan *bounding box* (Redmon dkk., 2016)..

Prinsip kerjanya sesuai dengan namanya, yaitu memproses seluruh piksel gambar secara bersamaan hanya dalam satu kali penelusuran. Dengan cara ini YOLO jadi sangat cepat dan mampu memahami konteks visual secara utuh tanpa perlu memotong gambar menjadi bagian-bagian kecil terlebih dahulu.

1.2. YOLO26 dan Penerapan pada Proyek

Pada proyek ini, kami menggunakan algoritma YOLO26 versi Nano (yolo26n) karena ukurannya yang ringan dan prosesnya cepat. Secara bawaan, YOLO26 sebenarnya hanya bertugas mendeteksi jenis dan lokasi objek. Agar sistem ini juga bisa mengukur jarak, kami memakai pendekatan Multi-Task Learning dengan menggabungkan detektor yolo26n dan modul pengukur kedalaman (depth head) di dalam satu file model (multitask_best.pt). Dengan cara ini, komputer hanya perlu membaca gambar satu kali untuk menjalankan dua tugas sekaligus: mendeteksi objek dan membuat peta kedalaman (depth map).

Untuk menentukan jarak objek ke kamera, sistem menggunakan teknik Region of Interest (ROI) Mapping. Caranya sederhana: kotak deteksi (bounding box) hasil YOLO akan dicocokkan lokasinya dengan area yang sama di depth map. Nilai kedalaman di area tersebut kemudian diambil dan diubah menjadi angka jarak nyata dalam satuan meter.

Dalam pengerjaannya, proyek ini memiliki beberapa batasan sistem:

1. **Objek Dibatasi 4 Kelas**: Sistem difokuskan untuk mendeteksi orang, botol, laptop, dan HP. Keempat objek ini

dipilih karena merupakan benda yang paling sering berada di depan kamera pada area dalam ruangan.

2. Output Aplikasi: Hasil akhir yang ditampilkan di layar berupa *bounding box*, label nama objek, tampilan visual *depth map*, serta nilai jarak objek dalam satuan meter.
3. Lingkungan Pengujian: Aplikasi dieksekusi dan diuji murni menggunakan laptop pada lingkungan Visual Studio berbasis PyTorch.



Sumber: https://developers.google.com/ar/develop/java/depth/quickstart?hl=id

Gambar 1.1 Ilustrasi umum RGB dan monocular depth pada perangkat mobile; bukan tangkapan layar atau hasil pengujian aplikasi proyek.

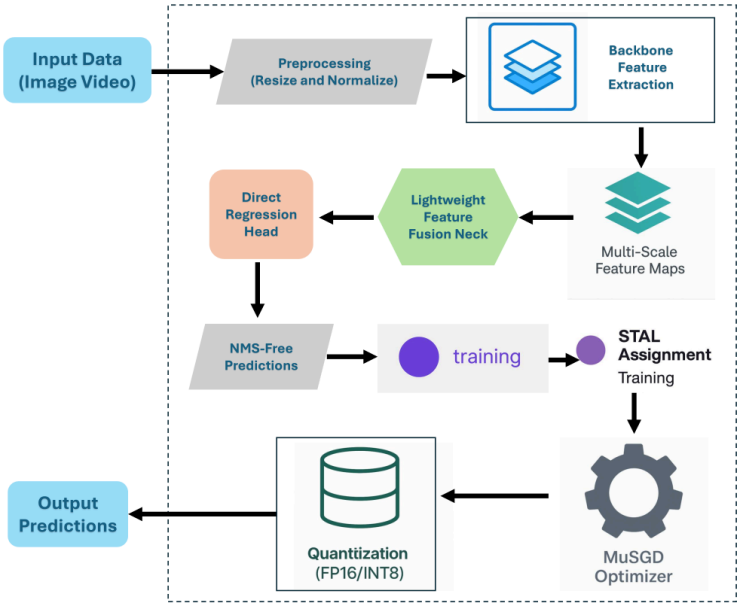
1.2.1 ARSITEKTUR YOLO V26

Secara sederhana, arsitektur YOLO26n terdiri dari tiga bagian utama:

- **Backbone**: Bertugas membaca dan memahami isi gambar, mulai dari garis, tekstur, hingga bentuk dasar objek.

- **Neck**: Menggabungkan informasi dari berbagai ukuran gambar agar objek berukuran kecil maupun besar dapat terdeteksi dengan baik.
- **Detection Head**: Mengolah hasil dari *Neck* untuk menebak nama objek sekaligus menentukan posisi kotaknya (*bounding box*) secara langsung.

Pada model *multitask* yang digunakan, hasil olahan dari bagian *Neck* dikirim ke dua saluran sekaligus: ke **Detection Head** untuk mengenali objek, dan secara bersamaan ke **Depth Head** untuk membuat peta kedalaman (*depth map*).



Sumber: https://learnopencv.com/yolov26-real-time-deployment/

Gambar 1.2 Referensi arsitektur YOLO26; bukan diagram lengkap depth head multitask runtime proyek.

1.3 DATASET

Pelatihan deteksi dan depth memerlukan citra serta label yang sesuai untuk tiap tugas. Dataset yang benar-benar digunakan harus mengikuti konfigurasi dan catatan run checkpoint, bukan daftar dataset contoh pada tinjauan pustaka.

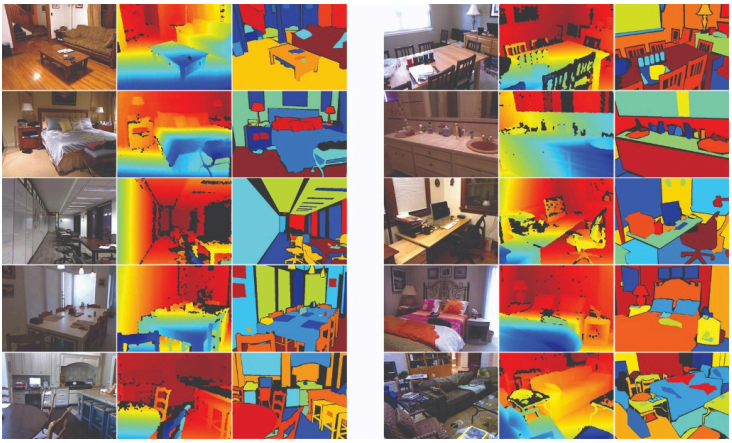
Metadata checkpoint multitask_best.pt mencatat LABEL_MODE=SUNRGBD, COCO_MIX=0.3, empat kelas person/laptop/cell phone/bottle, N_VAL=500, dan N_TEST=500. Metadata yang tersedia tidak cukup untuk mengklaim bahwa NYU Depth V2 atau KITTI dipakai pada run ini.

- SUN RGB-D: mode label depth yang dicatat pada konfigurasi checkpoint eksperimen ini. Detail subset dan pembagian berkas mengikuti konfigurasi/log run.

- COCO mix: konfigurasi run mencatat COCO_MIX=0.3 untuk campuran data deteksi. Ini bukan bukti bahwa COCO digunakan sebagai dataset depth atau detector runtime menggunakan kelas COCO lengkap.

- MS COCO (Common Objects in Context): dataset deteksi objek; konfigurasi run mencatat COCO_MIX=0.3. Bobot detector hasil akhir disesuaikan dengan empat kelas pada checkpoint.

Checkpoint berisi empat kelas target dan depth metric hasil pelatihan multitask. Klaim generalisasi ke semua lingkungan atau deployment smartphone memerlukan evaluasi tambahan.



Sumber: https://cs.nyu.edu/~fergus/datasets/nyu_depth_v2.html

Gambar 1.4 Contoh dari NYU Depth V2 sebagai ilustrasi dataset depth umum; bukan dataset atau hasil inferensi yang diklaim untuk checkpoint ini.

2.1 Deskripsi Alur Pipeline Baseline

Aplikasi kamera ini berjalan menggunakan satu berkas model multitask PyTorch (.pt) untuk mendeteksi objek sekaligus membuat peta kedalaman (depth map) dari webcam secara langsung. Seluruh proses inferensi dieksekusi murni pada lingkungan PyTorch.

Alur Kerja Program:

1. Ambil Gambar: Mengambil frame video dari webcam.
2. Olah Gambar: Mengatur ukuran gambar (letterbox) dan menyelaraskan nilai pikselnya.
3. Proses Model: Memproses deteksi objek dan peta kedalaman secara bersamaan di PyTorch.
4. Kembalikan Skala: Menyesuaikan kembali posisi kotak deteksi dan peta kedalaman ke ukuran layar asli.
5. Penyempurnaan: Menghaluskan tampilan kedalaman antar-frame dan menerapkan rumus kalibrasi.
6. Tampilkan Hasil: Menampilkan kotak deteksi, label objek, jarak dalam meter, dan visualisasi warna kedalaman pada layar.

2.2 Kode Program Baseline

Potongan kode berikut merupakan ringkasan fungsi utama dari aplikasi kamera real-time yang dijalankan melalui Visual Studio. Implementasi lengkapnya terbagi di dalam berkas multitask_pt.py dan main.py.

```
import cv2
# Memuat modul dari multitask_pt.py dan utils.py
from multitask_pt import load_multitask_torch_model
from utils import DepthEMASmoother, calibrate_depth_map,
get_depth_in_bbox, normalize_depth_for_display

# 1. Inisialisasi Model Multitask (.pt)
```

```
# Memuat checkpoint model PyTorch secara strict
(Backbone + Detection Head + Depth Head)
model = load_multitask_torch_model("multitask_best.pt")
smoother = DepthEMASmoother(alpha=0.2)

# --- Loop Utama Kamera (main.py) ---
while True:
   ret, frame = cap.read()
   if not ret:
      break

   # 2. Inferensi Multitask (Eksekusi di multitask_pt.py)
   # Menghasilkan bounding box [x1, y1, x2, y2, conf, cls] dan
depth map (meter)
   detections, depth_m = model.predict(frame)

   # 3. Smoothing & Kalibrasi Depth Map
   # Meredam fluktuasi antar-frame dan menerapkan
skala/offset empiris
   depth_smoothed = smoother.update(depth_m)
   depth_calibrated = calibrate_depth_map(depth_smoothed,
depth_scale=0.9935897436, depth_offset_m=-1.0)

   # 4. Pengukuran Jarak Objek (ROI Mapping)
   for det in detections:
      x1, y1, x2, y2, conf, cls_id = det
      # Mengambil median depth pada patch 15x15 piksel di
pusat bounding box
      distance_m = get_depth_in_bbox(
         depth_m=depth_calibrated,
         bbox=(x1, y1, x2, y2),
         patch_size=15
      )

   # 5. Konversi Visualisasi Heatmap Kedalaman (Khusus
Tampilan UI)
   depth_color = cv2.applyColorMap(
      normalize_depth_for_display(depth_calibrated),
      cv2.COLORMAP_INFERNO
```

```
    )

    # Tampilkan hasil pada layar (main.py)
    cv2.imshow("Multi-Task YOLO26n & Depth Estimation",
frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

Sumber: Penulis

2.3 Penjelasan Logika dan Fungsi Program

Bagian ini menjelaskan fungsi teknis dari setiap komponen utama dalam skrip multitask_pt.py dan main.py dengan kalimat yang lebih sederhana:

- **Pengolahan Awal (Preprocessing)**: Gambar dari kamera disesuaikan ukurannya (*letterbox* 480 × 640), diubah ke format warna RGB, dan diselaraskan nilai pikselnya agar siap dibaca oleh model.
- **Proses Model (Inferensi Multitask)**: Model menjalankan dua tugas sekaligus dalam satu waktu: deteksi objek YOLO26n (menghasilkan lokasi kotak dan nama objek) serta *Depth Head* (menghasilkan peta kedalaman dalam satuan meter).
- **Penyesuaian Skala (Postprocessing)**: Posisi kotak deteksi dan peta kedalaman dikembalikan ukurannya agar sesuai dengan tampilan asli layar kamera.
- **Penghalusan dan Kalibrasi**: Angka kedalaman dihaluskan antar-*frame* agar tidak goyang, lalu disesuaikan dengan rumus kalibrasi empiris agar pembacaan jaraknya lebih stabil.
- **Pengukuran Jarak Objek (ROI Mapping)**: Sistem mengambil nilai tengah (*median*) kedalaman pada area kotak 15 × 15 piksel di pusat objek untuk menentukan jarak fisik nyata dalam satuan meter.

- **Visualisasi Warna (Heatmap)**: Peta kedalaman diubah menjadi warna gradien *INFERNO*. Warna ini **hanya untuk tampilan visual di layar (UI)** agar nyaman dilihat dan tidak dipakai dalam perhitungan angka jarak.
- **Tampilan dan Kontrol**: Layar dapat menampilkan gambar secara berdampingan atau ditumpuk (*overlay*). Tombol **s** mengganti mode tampilan, **d** mengaktifkan *overlay*, **q** menghentikan program, dan klik *mouse* digunakan untuk membaca jarak di titik tertentu.
- **Pemuatan Model (Checkpoint)**: Fungsi load_multitask_torch_model() memastikan berkas bobot multitask_best.pt lengkap dan sesuai sebelum program kamera dijalankan.

-

3.1 Spesifikasi Arsitektur Baseline (YOLO26n Multitask)

Model yang dibangun berbasis arsitektur YOLO26n Multitask, yang mengombinasikan shared encoder (Backbone dan Neck) dengan dua head terpisah (Dual-Head), yaitu Detection Head dan Depth Head. Tabel 3.1 merangkum konfigurasi spesifikasi arsitektur baseline acuan beserta modul penyesuaian multitask dengan empat kelas target yang diimplementasikan pada sistem runtime.

Tabel 3.1 Ringkasan Konfigurasi Arsitektur Model Multitask YOLO26n

Komponen	Layer / Sumber	Modul Utama	Output / Fungsi
Compound Scaling	Varian YOLO26n	depth = 0.50, width = 0.25, max channels = 1024	Varian model nano yang efisien (260 layer, ~2.57M parameter, 6.1 GFLOPs) untuk eksekusi real-time pada perangkat desktop.
Backbone	Layer 0–10	Conv, C3k2, SPPF, C2PSA	Ekstraksi fitur hierarkis dari skala P1/2 hingga P5/32
Fitur P3	Layer 3–4 (digunakan kembali di Neck)	Conv dan C3k2	Menyediakan fitur spasial detail untuk deteksi objek berukuran kecil (small objects)
Fitur P4	Layer 5–6	Conv dan C3k2	Menyediakan

	(digunakan kembali di Neck)		fitur tingkat menengah untuk objek berukuran sedang (medium objects)
Fitur P5	Layer 7–10	Conv, C3k2, SPPF, C2PSA	Menyediakan fitur semantik tingkat tinggi untuk objek berukuran besar (large objects)
Neck	Layer 11–22	Upsample, Concat, Conv, C3k2	Menggabungkan (aggregate) fitur multi-skala dari alur backbone untuk diteruskan ke Dual-Head.
Detection Head	Layer 23 (dari Layer 16, 19, 22)	Detect	Memprediksi bounding box dan label klasifikasi 4 kelas target (person, bottle, laptop, cell phone) pada skala P3/8, P4/16, dan P5/32.
Depth Head	Percabangan dari Neck (P3,	Conv, Bilinear Interpolation, Feature Fusion	Mengolah fitur multi-skala menjadi depth

	P4, P5)		map satu kanal (satuan meter) untuk estimasi jarak objek.

Sumber: Ardhi et al. (2026) dan Pengembangan Penulis

3.1.1 Skala Komputasi (Compound Scaling)

Arsitektur YOLO26n dikonfigurasi menggunakan depth factor sebesar 0.50 dan width factor sebesar 0.25 dengan batas saluran maksimum 1024 channel. Konfigurasi ini menghasilkan model yang efisien dan ringan dengan total 260 layer, 2.572.280 parameter, serta beban komputasi sebesar 6.1 GFLOPS, sehingga ideal untuk dijalankan secara real-time pada lingkungan desktop.

3.1.2 Ekstraksi Fitur (Backbone):

Bagian backbone beroperasi pada Layer 0 hingga 10 menggunakan kombinasi konvolusi standar (Conv), blok C3k2, SPPF (Spatial Pyramid Pooling - Fast), dan C2PSA (Cross Stage Partial with Attention). Backbone mengekstrak representasi citra secara hierarkis yang terbagi menjadi tiga tingkatan resolusi:

- Fitur P3 (Layer 3–4): Mempertahankan informasi spasial detail yang berguna untuk mendeteksi objek berukuran kecil.
- Fitur P4 (Layer 5–6): Mengambil fitur tekstur tingkat menengah untuk objek berukuran sedang.
- Fitur P5 (Layer 7–10): Menangkap informasi semantik tingkat tinggi untuk membedakan objek berukuran besar.

Penjelasan fungsi modul penyusun *backbone*:

- **Conv (Convolution)**: Melakukan *downsampling* (memperkecil resolusi spasial) sekaligus mengekstrak fitur visual dasar.
- **C3k2**: Blok pemroses utama berbasis *Cross Stage Partial* (CSP) yang efisien mengekstrak fitur tanpa membengkakkan beban memori.
- **SPPF**: Menggabungkan peta fitur dari berbagai skala *pooling* secara cepat untuk memperluas area pandang (*receptive field*) model.
- **C2PSA**: Blok *attention* bawaan YOLO26n yang berfungsi menangkap hubungan spasial antarfitur sebelum diteruskan ke bagian Neck.

3.1.3 Penggabungan Fitur Multi-Skala (Neck)

Terletak pada Layer 11 hingga 22, komponen Neck memanfaatkan fungsi Upsample, Concat, Conv, dan C3k2 melalui arsitektur FPN (Feature Pyramid Network) + PAN (Path Aggregation Network). Neck menggabungkan peta fitur dari berbagai resolusi yang dihasilkan oleh backbone, memastikan bahwa fitur spasial detail dan fitur semantik kaya dapat saling melengkapi sebelum disalurkan ke Dual-Head.

3.1.4 Prediksi Akhir (Detection Head & Task Setting):

Sistem menggunakan pendekatan *Dual-Head* setelah bagian Neck:

1. **Detection Head**: Menerima fitur dari layer 16, 19, dan 22 pada skala P3/8, P4/16, dan P5/32 untuk menghasilkan prediksi *bounding box* dan skor klasifikasi secara *end-to-end* untuk 4 kelas target (*person*, *bottle*, *laptop*, *cell phone*).
2. **Depth Head**: Menerima peta fitur dari Neck untuk diproyeksikan dan digabungkan (*fusion*) menjadi peta

kedalaman (*depth map*) satu kanal beresolusi penuh dalam satuan meter aktual.

3.1.5 Alur Arsitektur dan Modifikasi Model

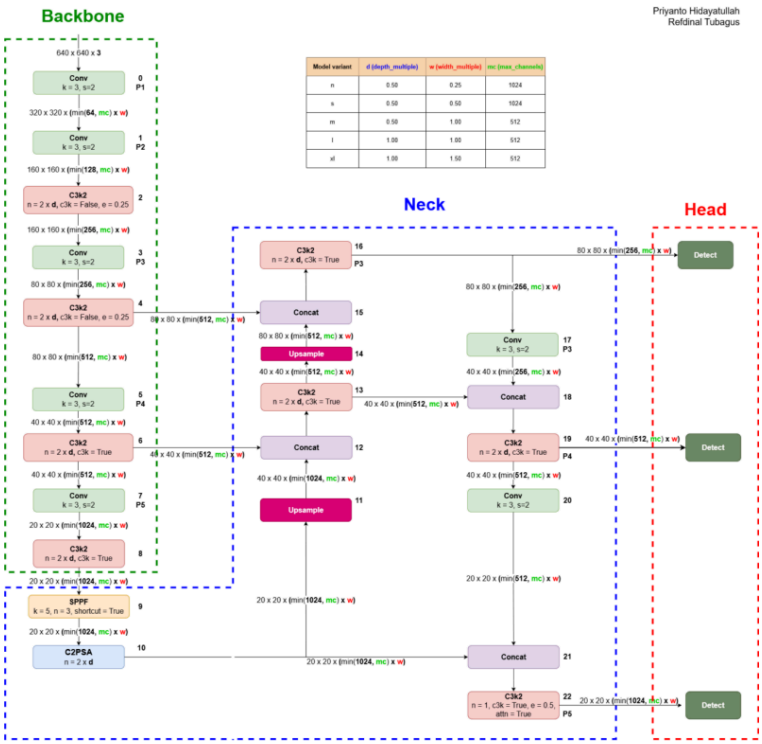
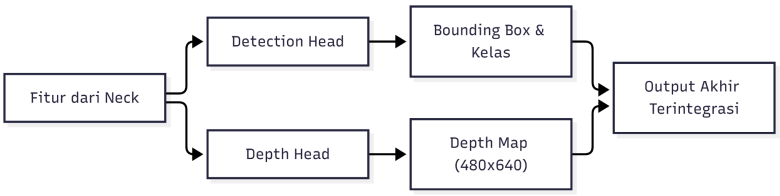


Figure 1: YOLO26 Architecture Diagram

Sumber: Hidayatullah dan Tubagus (2026)

Gambar 3.1 Arsitektur Utama YOLO26 (Backbone & Neck)



Sumber: Penulis

Gambar 3.2 Diagram Percabangan Dual-Head Model Multitask

Diagram arsitektur pada Gambar 3.1 menggambarkan alur perjalanan data dari citra masukan hingga menghasilkan dua jenis output secara serentak (multitask):

- Backbone (Layer 0–10):
    - Frame masukan *webcam* beresolusi asli diolah (*letterbox*) menjadi resolusi standar 480×640 piksel.
    - **Stem Layer (Layer 0–1)**: Menggunakan Konvolusi 3×3 (Conv) dengan *stride* 2 untuk mereduksi resolusi spasial awal sekaligus mengekstrak fitur visual dasar.
    - **Ekstraksi Hierarkis (Layer 2, 4, 6, 8)**: Memanfaatkan modul C3k2 untuk mengekstrak fitur visual abstrak secara efisien dari tingkatan skala $P2/4$ hingga $P5/32$.
    - **SPPF & C2PSA (Layer 9–10)**: Modul SPPF memperluas area pandang (*receptive field*) fitur, diikuti oleh modul *self-attention* bawaan C2PSA untuk memperkuat hubungan spasial antarfitur semantik sebelum disalurkan ke Neck.
- Neck / FPN + PAN (Layer 11–22) :
    - Komponen Neck menggabungkan peta fitur dari *backbone* melalui dua alur utama: *Feature*

Pyramid Network (FPN) secara *top-down* dan *Path Aggregation Network* (PAN) secara *bottom-up*.

- Proses penggabungan (*concat*) dan penyesuaian ukuran (*upsample*/*downsample*) menghasilkan tiga tingkatan peta fitur kaya informasi spasial dan semantik, yaitu pada skala $P3/8$ (deteksi kecil), $P4/16$ (deteksi sedang), dan $P5/32$ (deteksi besar).

Gambar 3.2 menjelaskan bagaimana tiga tingkatan peta fitur ($P3, P4, P5$) keluaran dari *Neck* dialirkan secara bersamaan (*simultaneous forward pass*) ke dua modul *head* independen:

- Detection Head:
  - Menerima peta fitur multi-skala dari Neck untuk memprediksi lokasi *bounding box* (x_1, y_1, x_2, y_2), skor *confidence*, dan probabilitas klasifikasi untuk 4 kelas target (*person*, *bottle*, *laptop*, *cell phone*).
  - Eksekusi dilakukan secara *end-to-end* (bebas operasi NMS) untuk memastikan latensi inferensi tetap rendah.
- Depth Head:
  - Menerima peta fitur dari Neck untuk diproyeksikan (256 kanal), digabungkan (*feature fusion*), dan disesuaikan ukurannya melalui interpolasi *bilinear*.
  - Menghasilkan *depth map* satu kanal beresolusi penuh (480×640) dengan nilai kedalaman fisik langsung dalam satuan meter metrik.
- Output Akhir Terintegrasi:

- Hasil olahan *bounding box* dan *depth map* digabungkan di tingkat aplikasi *runtime*.
- Sistem melakukan *ROI Mapping* (15 × 15 *patch* di pusat *bounding box*) untuk mengekstrak nilai jarak metrik objek, lalu menampilkan *heatmap* kedalaman beserta label deteksi pada antarmuka *webcam* secara *real-time*.

3.2 Penjelasan Kesesuaian Program dengan Arsitektur

Subbab ini menguraikan struktur modul PyTorch dan blok-blok logika yang membentuk arsitektur jaringan YOLO26n Multitask. Seluruh modul dirancang secara teratur untuk mendukung eksekusi Shared Encoder (Backbone dan Neck) hingga percabangan keluaran Dual-Head (Detection Head dan Depth Head).

1. ConvBlock

ConvBlock merupakan blok pembangun dasar yang digunakan di seluruh bagian arsitektur. Blok ini terdiri atas konvolusi 2D, Batch Normalization, dan fungsi aktivasi SiLU (Sigmoid Linear Unit). Konvolusi mengekstrak karakteristik visual dasar, Batch Normalization menjaga kestabilan distribusi nilai fitur, sedangkan aktivasi SiLU memberikan kemampuan non-linearitas agar model dapat mempelajari representasi fitur yang kompleks.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

def autopad(k, p=None, d=1):
  if d > 1:
    k = d * (k - 1) + 1 if isinstance(k, int) else [d * (x - 1) + 1
for x in k]
  if p is None:
    p = k // 2 if isinstance(k, int) else [x // 2 for x in k]
```

```
    return p

class ConvBlock(nn.Module):
    def __init__(self, c1, c2, k=1, s=1, p=None, g=1, d=1,
act=True):
        super().__init__()
        self.conv = nn.Conv2d(c1, c2, k, s, autopad(k, p, d),
groups=g, dilation=d, bias=False)
        self.bn = nn.BatchNorm2d(c2)
        self.act = nn.SiLU() if act is True else (act if
isinstance(act, nn.Module) else nn.Identity())

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.act(self.bn(self.conv(x)))
```

2. Bottleneck

Modul Bottleneck terdiri atas dua lapisan ConvBlock berurutan dengan koneksi residual (shortcut). Koneksi residual memungkinkan informasi fitur awal diteruskan secara langsung ke output, sehingga mempermudah aliran gradien saat pelatihan dan mencegah masalah vanishing gradient. Blok ini menjadi komponen utama penyusun modul C3k2.

```
class Bottleneck(nn.Module):
    def __init__(self, c1, c2, shortcut=True, g=1, k=(3, 3),
e=0.5):
        super().__init__()
        c_ = int(c2 * e)
        self.cv1 = ConvBlock(c1, c_, k[0], 1)
        self.cv2 = ConvBlock(c_, c2, k[1], 1, g=g)
        self.add = shortcut and c1 == c2

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return x + self.cv2(self.cv1(x)) if self.add else
self.cv2(self.cv1(x))
```

3. C3k2

Modul C3k2 merupakan blok ekstraksi fitur berbasis Cross Stage Partial (CSP). Fitur masukan dibagi menjadi dua jalur: satu jalur diproses langsung melalui konvolusi, sedangkan jalur lainnya diproses melalui deretan blok Bottleneck. Kedua jalur kemudian digabungkan (concatenation) pada dimensi channel (dim=1) dan diproyeksikan kembali, sehingga menghasilkan representasi fitur yang kaya tanpa membengkakkan beban komputasi.

```
class C3k2(nn.Module):
   def __init__(self, c1, c2, n=1, c3k=False, e=0.5, g=1,
shortcut=True):
      super().__init__()
      c_ = int(c2 * e)
      self.cv1 = ConvBlock(c1, c_, 1, 1)
      self.cv2 = ConvBlock(c1, c_, 1, 1)
      self.m = nn.ModuleList([Bottleneck(c_, c_, shortcut, g,
k=(3, 3), e=1.0) for _ in range(n)])
      self.cv3 = ConvBlock(2 * c_, c2, 1, 1)

   def forward(self, x: torch.Tensor) -> torch.Tensor:
      y1 = self.cv1(x)
      y2 = self.cv2(x)
      for m in self.m:
         y2 = m(y2)
      return self.cv3(torch.cat((y1, y2), dim=1))
```

4. C2PSA

C2PSA (Cross Stage Partial with Attention) merupakan modul attention bawaan (stock) pada arsitektur YOLO26n. Modul ini menerapkan Adaptive Average Pooling dan konvolusi berbobot sigmoid untuk memberikan pembobotan antar channel (mekanisme spatial & channel attention), sehingga model fokus pada area fitur yang paling relevan sebelum disalurkan ke bagian Neck.

```
class C2PSA(nn.Module):
   def __init__(self, c1, c2, n=1, e=0.5):
      super().__init__()
      c_ = int(c2 * e)
      self.cv1 = ConvBlock(c1, 2 * c_, 1, 1)
      self.cv2 = ConvBlock(2 * c_, c2, 1, 1)
      self.attn = nn.Sequential(
         nn.AdaptiveAvgPool2d(1),
         nn.Conv2d(c2, c2 // 4, 1),
         nn.SiLU(),
         nn.Conv2d(c2 // 4, c2, 1),
         nn.Sigmoid()
      )

   def forward(self, x: torch.Tensor) -> torch.Tensor:
      x = self.cv1(x)
      return self.cv2(x * self.attn(x))
```

5. SPPF

Modul SPPF berfungsi untuk memperluas area pandang (receptive field) model secara efisien. Dengan melakukan operasi max pooling bertahap menggunakan ukuran kernel konstan (5 x 5), SPPF mampu menangkap informasi konteks multi-skala dan menggabungkannya dengan fitur awal tanpa menurunkan resolusi spasial.

```
class SPPF(nn.Module):
   def __init__(self, c1, c2, k=5):
      super().__init__()
      c_ = c1 // 2
      self.cv1 = ConvBlock(c1, c_, 1, 1)
      self.cv2 = ConvBlock(c_ * 4, c2, 1, 1)
      self.m = nn.MaxPool2d(kernel_size=k, stride=1,
padding=k // 2)

   def forward(self, x: torch.Tensor) -> torch.Tensor:
      x = self.cv1(x)
```

```
        y1 = self.m(x)
        y2 = self.m(y1)
        y3 = self.m(y2)
        return self.cv2(torch.cat((x, y1, y2, y3), dim=1))
```

6. Backbone

Backbone bertanggung jawab melakukan ekstraksi fitur hierarkis dari gambar masukan. Proses dimulai dari konvolusi awal (stem) dengan stride 2, diikuti oleh kombinasi modul ConvBlock, C3k2, SPPF, dan C2PSA. Backbone menghasilkan tiga tingkatan peta fitur utama:

- P3: Resolusi spasial tinggi untuk deteksi objek berukuran kecil.
- P4: Resolusi menengah untuk deteksi objek berukuran sedang.
- P5: Resolusi spasial rendah dengan pemahaman semantik tingkat tinggi untuk objek besar.

```
class Backbone(nn.Module):
    def __init__(self, c_stem=16, c_p2=32, c_p3=64, c_p4=128,
c_p5=256, n_repeat=1):
        super().__init__()
        self.stem = ConvBlock(3, c_stem, k=3, s=2)
        self.l2 = C3k2(c_stem, c_p2, n=n_repeat)
        self.l3 = ConvBlock(c_p2, c_p3, k=3, s=2)
        self.l4 = C3k2(c_p3, c_p3, n=n_repeat)
        self.l5 = ConvBlock(c_p3, c_p4, k=3, s=2)
        self.l6 = C3k2(c_p4, c_p4, n=n_repeat)
        self.l7 = ConvBlock(c_p4, c_p5, k=3, s=2)
        self.l8 = C3k2(c_p5, c_p5, n=n_repeat)
        self.l9 = SPPF(c_p5, c_p5, k=5)
        self.l10 = C2PSA(c_p5, c_p5)

    def forward(self, x: torch.Tensor):
        x = self.stem(x)
```

```
        x = self.l2(x)
        x = self.l3(x)
        p3 = self.l4(x)
        x = self.l5(p3)
        p4 = self.l6(x)
        x = self.l7(p4)
        x = self.l8(x)
        x = self.l9(x)
        p5 = self.l10(x)
        return p3, p4, p5
```

7. Neck FPN dan PAN

Komponen Neck menggabungkan fitur multi-skala dari backbone menggunakan gabungan arsitektur Feature Pyramid Network (FPN) dan Path Aggregation Network (PAN). Alur FPN menyebarkan informasi semantik secara top-down (P5 ke P3), sedangkan PAN menyalurkan kembali informasi posisi spasial presisi secara bottom-up (P3 ke P5). Hasil akhirnya berupa tiga peta fitur kaya konteks (p3_out, p4_out, p5_out).

```
class Neck(nn.Module):
    def __init__(self, c_p3=64, c_p4=128, c_p5=256):
        super().__init__()
        self.up_p5_to_p4 = nn.Upsample(scale_factor=2,
mode='nearest')
        self.cv_p4 = C3k2(c_p5 + c_p4, c_p4)
        self.up_p4_to_p3 = nn.Upsample(scale_factor=2,
mode='nearest')
        self.cv_p3 = C3k2(c_p4 + c_p3, c_p3)

        self.down_p3_to_p4 = ConvBlock(c_p3, c_p3, k=3, s=2)
        self.pan_p4 = C3k2(c_p3 + c_p4, c_p4)
        self.down_p4_to_p5 = ConvBlock(c_p4, c_p4, k=3, s=2)
        self.pan_p5 = C3k2(c_p4 + c_p5, c_p5)

    def forward(self, p3, p4, p5):
        f_p5_up = self.up_p5_to_p4(p5)
```

```
        p4_fused = self.cv_p4(torch.cat([f_p5_up, p4], dim=1))
        f_p4_up = self.up_p4_to_p3(p4_fused)
        p3_out = self.cv_p3(torch.cat([f_p4_up, p3], dim=1))

        p3_down = self.down_p3_to_p4(p3_out)
        p4_out = self.pan_p4(torch.cat([p3_down, p4_fused],
dim=1))
        p4_down = self.down_p4_to_p5(p4_out)
        p5_out = self.pan_p5(torch.cat([p4_down, p5], dim=1))

        return p3_out, p4_out, p5_out
```

8. Detection Head

DetectionHead bertugas memprediksi *bounding box* dan label kelas objek pada tiga skala berbeda ($P3/8$, $P4/16$, $P5/32$). Setiap *head* memiliki dua cabang konvolusi terpisah: cabang regresi (reg) untuk memprediksi koordinat lokasi (x_1, y_1, x_2, y_2) dan cabang klasifikasi (cls) untuk menentukan probabilitas 4 kelas target (*person*, *bottle*, *laptop*, *cell phone*).

```
class DetectionHead(nn.Module):
    def __init__(self, in_ch: int, num_classes: int = 4):
        super().__init__()
        self.reg = nn.Sequential(
            ConvBlock(in_ch, in_ch, k=3, s=1),
            nn.Conv2d(in_ch, 4, kernel_size=1)
        )
        self.cls = nn.Sequential(
            ConvBlock(in_ch, in_ch, k=3, s=1),
            nn.Conv2d(in_ch, num_classes, kernel_size=1)
        )

    def forward(self, x: torch.Tensor):
        return self.reg(x), self.cls(x)
```

9. DepthHead

DepthHead memproyeksikan peta fitur multi-skala dari Neck ($P3, P4, P5$) ke jumlah *channel* yang setara, lalu menyesuaikan ukurannya ke dimensi target menggunakan interpolasi *bilinear*. Ketiga peta fitur digabungkan (*fusion*) dan diproses oleh blok regresi untuk menghasilkan peta kedalaman (*depth map*) satu kanal dengan nilai meter metrik aktual.

```
class DepthHead(nn.Module):
   def __init__(self, channels=(64, 128, 256), out_ch=32):
      super().__init__()
      self.conv_p3 = ConvBlock(channels[0], out_ch, k=1, s=1)
      self.conv_p4 = ConvBlock(channels[1], out_ch, k=1, s=1)
      self.conv_p5 = ConvBlock(channels[2], out_ch, k=1, s=1)
      self.refine = nn.Sequential(
         ConvBlock(out_ch * 3, 64, k=3, s=1),
         nn.Conv2d(64, 1, kernel_size=1)
      )

   def forward(self, p3, p4, p5, target_size):
      up_p3 = F.interpolate(self.conv_p3(p3), size=target_size,
mode='bilinear', align_corners=False)
      up_p4 = F.interpolate(self.conv_p4(p4), size=target_size,
mode='bilinear', align_corners=False)
      up_p5 = F.interpolate(self.conv_p5(p5), size=target_size,
mode='bilinear', align_corners=False)
      fused = torch.cat([up_p3, up_p4, up_p5], dim=1)
      return self.refine(fused)
```

10. Forward Model

Fungsi forward() mengatur alur pemrosesan terintegrasi dari masukan citra hingga keluaran Dual-Head. Gambar masukan pertama kali diekstraksi oleh Backbone, digabungkan oleh Neck, lalu disalurkan secara paralel ke tiga modul DetectionHead dan satu modul DepthHead.

```
class YOLO26Multitask(nn.Module):
```

```
  def __init__(self, num_classes=4):
    super().__init__()
    self.backbone = Backbone()
    self.neck = Neck()
    self.det_p3 = DetectionHead(64, num_classes)
    self.det_p4 = DetectionHead(128, num_classes)
    self.det_p5 = DetectionHead(256, num_classes)
    self.depth_head = DepthHead(channels=(64, 128, 256))

  def forward(self, x: torch.Tensor):
    target_size = (x.shape[2], x.shape[3])

    p3, p4, p5 = self.backbone(x)
    p3_out, p4_out, p5_out = self.neck(p3, p4, p5)

    p3_reg, p3_cls = self.det_p3(p3_out)
    p4_reg, p4_cls = self.det_p4(p4_out)
    p5_reg, p5_cls = self.det_p5(p5_out)
    depth_map = self.depth_head(p3_out, p4_out, p5_out,
target_size)

    return {
      'p3_bbox': p3_reg, 'p3_cls': p3_cls,
      'p4_bbox': p4_reg, 'p4_cls': p4_cls,
      'p5_bbox': p5_reg, 'p5_cls': p5_cls,
      'depth': depth_map
    }
```

11. Output Model

Model mengembalikan luaran dalam bentuk kamus (*dictionary*) PyTorch Tensor yang berisi hasil prediksi lokasi objek, skor kelas, dan estimasi kedalaman secara terpisah:

- p3_bbox, p4_bbox, p5_bbox: Tensor regresi lokasi *bounding box* pada skala $P3$, $P4$, $P5$.

- p3_cls, p4_cls, p5_cls: Tensor skor klasifikasi untuk 4 kelas target (*person*, *bottle*, *laptop*, *cell phone*).
- depth: Tensor peta kedalaman satu kanal beresolusi 480×640 piksel dengan estimasi jarak metrik dalam satuan meter.

Struktur keluaran terpisah ini memungkinkan aplikasi *runtime* mengeksekusi post-processing deteksi dan *ROI Mapping* kedalaman secara efisien dalam satu kali *forward pass*.

4.1 Evaluasi Kinerja Kuantitatif dan Analisis Visualisasi Output



Sumber: Penulis; contoh peta depth bersifat kualitatif.

Gambar 4.1 Hasil Output Depth Map

Tabel 4.1 Metrik Validasi yang Tersimpan pada Checkpoint Multitask (Epoch 10)

Metrik Evaluasi	Nilai Validasi	Keterangan
delta1	0,7803 (78,03%)	Proporsi piksel valid dengan rasio kedalaman < 1,25 pada validasi checkpoint.
Abs Rel	0,1586 (15,86%)	Rata-rata kesalahan relatif kedalaman; nilai lebih kecil lebih baik.
RMSE	0,5989 m	Root mean squared error depth pada validasi checkpoint.
mAP50-95	0,1299 (12,99%)	Metrik deteksi mean Average Precision pada rentang IoU 0,50-0,95.

Sumber: metadata checkpoint multitask_best.pt, epoch 10; run 20261013-163528-539afa.

4.2 Analisis Kelemahan dan Keterbatasan Model Baseline

4.3 Rekomendasi Solusi dan Arah Perbaikan

4.4 Analisis Komparatif dan Keterbatasan Arsitektur Kustom



Sumber: Penulis

Gambar 4.2 Perbandingan Visual Output Inferensi Real-Time Model Kustom (Atas) dan Model Bawaan (Bawah)